

《ADD 3.0 — 属性驱动设计方法》完整知识点整理

课件来源：南京大学软件学院 李杉杉 — Designing Software Architectures: A Practical Approach
课件页数：129页
关联考试：2026年期末考试 QI-5（ADD设计过程步骤）、QI-7（为什么需要不同视图）、QII-1（七类设计决定与绑定时间）等多道题直接相关

🔒 ADD 3.0 是目前工业界最成熟的系统化架构设计方法之一。它告诉你如何从驱动因子出发，通过迭代式设计，逐步构建出满足质量属性的架构。

目录

- 一、架构设计基础
- 二、架构驱动因子 (Architectural Drivers)
- 三、设计策略 (Design Strategies)
- 四、设计概念 (Design Concepts)
- 五、ADD 3.0 方法详解
- 六、应用ADD到不同系统场景
- 七、架构文档化
- 八、讨论问题与思考

一、架构设计基础

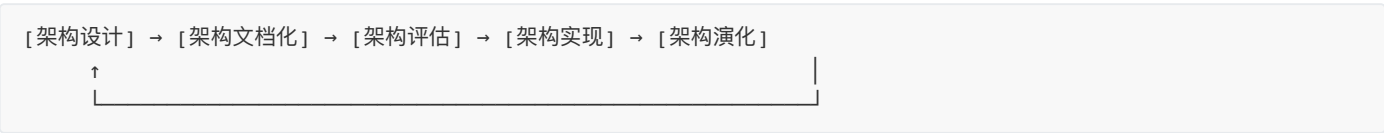
1.1 什么是软件架构？

《Software Architecture in Practice》定义：
The software architecture of a system is **the set of structures needed to reason about the system**. These structures comprise software elements, relations among them, and properties of both.

翻译：软件架构是推理系统所需的一组结构。这些结构包含软件元素、元素之间的关系，以及两者的属性。

1.2 架构生命周期活动

架构不是一次性的活动，而是贯穿系统生命周期的：



本课程的核心聚焦点是 **Architectural Design Activity**（架构设计活动）。

1.3 为什么需要设计方法？

1. 架构设计出奇地难以掌握 — 需要同时考虑大量因素，要求丰富的领域知识
2. 设计可以（也应该）被系统化执行 — 确保决策是基于驱动因子的、可追溯的、可被问责的
3. 为缺乏经验的人提供指导 — 避免架构设计变成"大师的神秘活动"

A design method combines the design process with the elements, relationships, and attributes to construct the desired architecture.

二、架构驱动因子（Architectural Drivers）

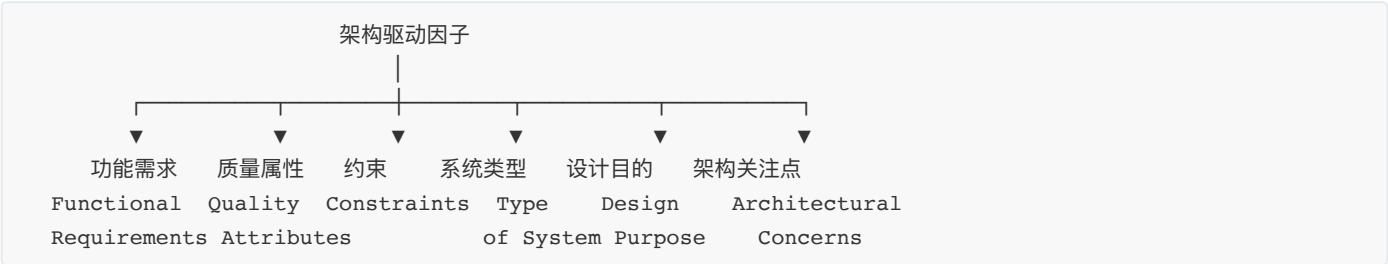
★ 对应考试 QI-1：软件需求、质量属性和架构显著需求（ASR）的区别与关系。

2.1 什么是驱动因子？

Architectural drivers are a subset of the requirements that shape the architecture — they are the inputs to the design process.

架构驱动因子是那些塑造架构形状的需求子集。它们是设计过程的输入。

2.2 六类驱动因子



#	驱动因子	英文	说明
1	功能需求	Functional Requirements	支持业务目标的主要功能
2	质量属性需求	Quality Attribute Requirements	可度量的系统特性（性能、可用性等），可用场景技术精确定义
3	约束	Constraints	不可协商的设计限制（时间、预算、技术、法规）
4	系统类型	Type of System	决定了设计的起点和可用的参考架构
5	设计目的	Design Purpose/Objectives	为什么而设计（售前方案 vs 定制系统 vs 持续演化系统的增量）
6	架构关注点	Architectural Concerns	额外的架构考量（日志、API版本管理、异常处理等）

2.3 功能需求作为驱动因子

- 通常涉及支持业务目标的主要功能（Primary Functionality）
- 例如：网管系统的"Marketecture"功能分解

2.4 质量属性作为驱动因子

- 是用户和开发者关心的可度量特性：性能、可用性、可修改性、可测试性等
- 使用质量属性场景技术进行精确描述
- 需要双重优先级排序：
 - 客户按对系统成功的重要性排序 (H/M/L)
 - 架构师按技术风险排序 (H/M/L)

2.5 约束（Constraints）

类比：建筑设计中，预算和土地面积限制了设计选择。

约束类型	说明	例子
时间与预算	项目截止日期限制设计复杂度	3个月项目 → 不能用需要6个月开发的复杂模块
技术限制	团队能力限制技术选择	团队不懂区块链 → 不加区块链子需求
业务规则与法规	必须满足的法律合规要求	电商系统必须遵守GDPR

2.6 系统类型

类型	英文	特征	例子
新领域绿地系统	Greenfield in novel domains	领域知识较少，更具创新性	Google早期、Amazon早期、WhatsApp
成熟领域绿地系统	Greenfield in mature domains	领域知识丰富，创新性较低	传统企业应用、标准移动应用
棕地系统	Brownfield systems	对已有系统的修改和扩展	遗留系统改造、功能增强

 **Greenfield** = 从零开始，在"绿地"上建造。 **Brownfield** = 在已有的"棕地"上改造。

2.7 设计目的

设计目的	特征
售前方案	快速设计初始方案以做出估算
定制系统	已确定时间和成本，发布后变化不大
持续演化系统的新增量	为新版本或发布做设计，架构需持续适应变化

2.8 架构关注点

不是传统需求，但也需要在架构设计中考虑：

类型	说明	例子
一般关注点	广泛的系统级问题	整体系统结构、功能→模块映射、模块→团队映射、代码库组织
具体关注点	内部细节问题（来自之前的决策）	日志策略、API版本管理
内部需求	不是来自客户但至关重要	特定加密方案、技术选择约束
问题/风险	需要架构变更来应对	安全风险、性能瓶颈

三、设计策略（Design Strategies）

3.1 分解（Decomposition）

将质量属性驱动因子分解并分配到分解出的各个元素上。

- 类比：把1000块的拼图分成"天空""山脉""人物"几个区域
- 目的：让抽象的需求变得可操作，通过为每个模块/组件分配明确的职责

- 需要时刻注意约束条件，让分解能容纳这些约束

设计活动的目标：产生一个满足约束、达成质量和业务目标的设计。

3.2 面向ASR设计（Designing to ASRs）

ASR = Architecturally Significant Requirements（架构显著需求）

ASR 的选择意味着需求的优先级排序。对于非ASR需求有三种情况：

情况	处理方式
A) 当前设计仍能满足	不需要额外处理
B) 稍作调整就能满足	轻微调整当前设计
C) 当前设计无法满足	a) 接近满足 → 微调；b) 重新调整优先级并重审设计；c) 确实无法满足 → 承认限制

🧠 一次设计所有ASR还是逐个设计？ — 这取决于经验。随着经验增长，你会发展设计直觉，能同时运用模式/战术来处理多个ASR。

3.3 生成与测试（Generate and Test）

将特定设计视为一个假设：当前假设中错误的部分在下一个假设中被修复，正确的部分被保留。

问题	答案
初始假设从哪来？	已有系统、框架（部分设计）、模式和战术、设计检查清单
用什么测试？	分析技术、设计检查清单
如何生成下一个假设？	修复当前假设的不足
何时结束？	设计满足所有ASRs，或设计预算用尽

四、设计概念（Design Concepts）

★ 对应考试 QI-4：区分模式和战术；QI-5：ADD Step 4 中如何选择设计概念。

4.1 核心思想：不要重新发明轮子

Architects are problem solvers, not inventors.

- 大多数子问题可以用已有的解决方案（设计概念）来解
- 使用经过验证的方法通常产出更好、更快、风险更低的设计
- 设计中的创造力体现在：明智地选择、聪明地组合、灵活地演化

4.2 五大类设计概念

设计概念（Design Concepts）

— 参考架构（Reference Architectures）

提供应用结构化的蓝图

例：移动应用、富客户端、Web应用、服务应用

拓展阅读：《Application Architecture Guide》

- 部署模式 (Deployment Patterns)
从物理角度构建系统的指导
例：2/3/4/n层部署、负载均衡集群、故障转移集群、私有/公有云
良好部署决策对可用性等QA至关重要
- 架构/设计模式 (Architectural/Design Patterns)
对反复出现设计问题的验证过的概念性解决方案
源自建筑学领域
难以严格区分"设计模式"和"架构模式"
例：三模冗余模式 (Tri-modular Redundancy)
- 战术 (Tactics)
影响质量属性响应控制的单一设计决策
针对7个质量属性的战术分类：可用性、互操作性、可修改性、性能、安全、可测试性、易用性
- 外部开发组件 (Externally Developed Components)
可复用的代码解决方案（中间件、框架等）
Framework：提供通用功能、处理跨应用关注点的可复用软件元素
例（Java）：Swing(UI)、Spring(组件连接)、Spring-Security(安全)、Hibernate(ORM)

4.3 设计概念选择路线图

针对绿地系统的设计概念选择顺序：

参考架构(Reference Architecture) → 部署模式(Deployment Patterns)
→ 架构模式(Architectural Patterns) → 战术(Tactics)
→ 外部组件(Externally Developed Components)

在处理质量属性驱动的迭代时，建议从战术出发，再上升到模式或技术。（ADD Iteration 3的实践：start with tactics and from there go to patterns or technologies）

4.4 设计概念选择方法

方法	说明	成本
优缺点表 (Pros-Cons Table)	列出每个备选概念的优缺点，支持基于驱动因子的选择	低
CBAM	定量方法：使用效用-响应曲线、成本效益分析和ROI	中
SWOT	评估备选方案的优势、劣势、机会和威胁	中
原型验证	当分析不够时，创建丢弃型原型。成本是分析的5-10倍	高

⚠ 原型验证的使用场景：项目涉及公司不熟悉的新技术、不确定的技术集成。不要轻易使用，成本远高于分析。

五、ADD 3.0 方法详解

★ 对应考试 Q1-5（6分简答题）：给出架构设计主要过程步骤，说明每步的输入和输出。

5.1 ADD的发展历史

版本	时间	特征与局限
ADD 1.0	2000.01	首个聚焦质量属性和架构结构/视图的设计方法，但缺少具体实现指导
ADD 2.0	2006.11	通过结合模式和战术链接QA与设计选择。在输入、元素分解、驱动因子和设计概念方面存在局限
ADD 3.0	2016	引入应用框架，从驱动因子→设计决策→可用实现选项。快速迭代，少量设计决策。促进参考架构的重用和"设计概念目录"

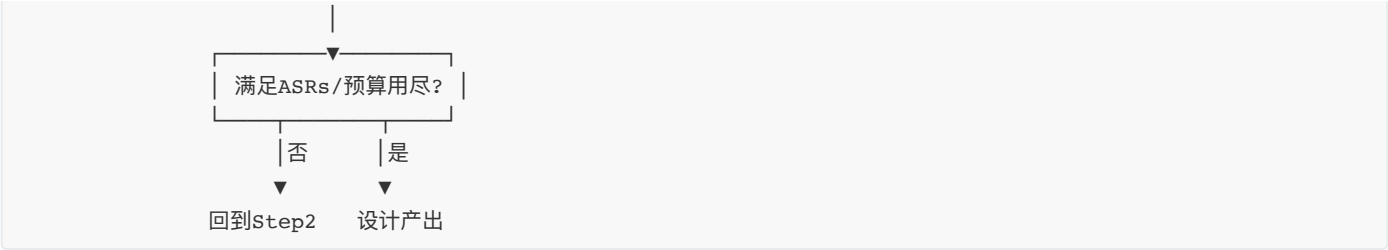
5.2 ADD 3.0 的核心特征

- 1. 迭代式设计方法：支持在项目增量（如Sprint）内进行结构化设计的微迭代
- 2. 稳定且被行业验证：经历开发实践变迁仍保持不变，证明其通用适用性
- 3. 适应性与一致性：适用于DevOps和云原生架构等现代技术，无需修改核心步骤
- 4. 非BDUF：产出的可能是部分架构设计，不是"Big Design Up Front"大设计

5.3 ADD 3.0 八步流程

ADD 的一个设计轮次（Design Round）包含一系列迭代，每个迭代聚焦特定目标。





5.4 每一步的详细解读

Step 1：评审输入（Review Inputs）

维度	内容
要确认的输入	设计目的、主要功能需求、质量属性场景、架构约束、架构关注点、已有的架构设计
关键活动	确认驱动因子已被正确排序（最好由最重要的涉众完成）
排序工具	Quality Attribute Workshop、Utility Tree
核心原则	避免 Garbage in, Garbage out — 驱动因子不对，设计全废
检查要点	是否遗漏重要涉众？业务条件是否变化？

输入：原始需求、约束、关注点
输出：经过验证和优先级排序的驱动因子列表

Step 2：确定迭代目标（Establish Iteration Goal）

维度	内容
核心思想	设计问题被分解为多个子问题，每次迭代解决一个子问题
目标大小	目标要“合适大小”（right-sized），不能太大也不能太小

三种合适的目标范围：

范围	例子
单个重要驱动因子	满足一个具有挑战性的QA场景 / 分解系统的架构关注点
一组相似驱动因子	一组相关的Use Case / 一组相关的QA场景
一组关联驱动因子	一个用户故事 + 一个关联的QA场景

案例系统的迭代目标设定：

迭代	目标	考虑的输入
Iteration 1	创建整体系统结构（Create an overall system structure）	全部驱动因子
Iteration 2	识别支持主要功能的结构（Identify elements supporting primary functionality）	主要用例（Primary Use Cases）
Iteration 3	满足QA-3可用性场景（系统故障后30秒内恢复运行）	QA-3可用性场景

输入：驱动因子及优先级
输出：明确的迭代目标声明

Step 3：选择要细化的元素（Choose Elements to Refine）

维度	内容
三种决策类型	分解为更细粒度元素（自顶向下）、合并为更粗粒度元素（自底向上）、改进已有元素
Greenfield起点	建立系统上下文，选择唯一可用元素（系统本身）进行分解
Brownfield/后期迭代	细化之前迭代中识别出的元素，需要对系统元素有充分理解

细化方式：

细化（Refinement）=

├─ 分解（Decomposition）

├─ 合并（Combination）

└─ 改进（Improvement）

— 自顶向下，大元素 → 小元素

— 自底向上，小元素 → 大元素

— 优化已有元素

📌 Step 2 和 Step 3 的顺序有时可以互换。

案例应用：

- Iteration 1：Greenfield系统，唯一元素 = 系统本身
- Iteration 2：细化的元素 = 第一轮迭代中识别的各个层
- Iteration 3：场景涉及整个系统故障，选择的元素 = 第一轮迭代中识别的各层

输入：当前架构设计 + 迭代目标
输出：被选中进行细化的元素

Step 4：选择设计概念（Choose Design Concepts）

维度	内容
核心原则	架构师是问题解决者，不是发明家。避免"重新发明轮子"

4.1 识别设计概念（Identify）：

概念类型	备选方案
参考架构	Mobile、Rich Client、Rich Internet、Service、Web 应用
部署模式	2-tier、3-tier、4-tier、负载均衡集群、故障转移集群

4.2 选择设计概念（Selection）：

方法	适用场景	成本
优缺点表	常规决策，基于驱动因子比较优劣	低
CBAM（成本效益分析）	需要精确定量分析的重大决策	中
SWOT分析	需要全面评估备选方案的优劣势和风险	中
原型验证	新技术/不确定技术整合	高（成本是分析的5-10倍）

案例应用：

- Iteration 1：选择 3-tier 部署模式 + 两种参考架构
- Iteration 2：选择 Domain Objects + 外部组件（框架）
- Iteration 3：从战术开始，再上升到模式或技术（start with tactics → patterns/technologies）

输入：迭代目标 + 被选中的元素

输出：选定的设计概念

Step 5：实例化元素、分配职责、定义接口

维度	内容
做什么	将选定的设计概念具体化为实际的架构元素
三个子任务	创建元素 → 分配职责 → 定义接口

案例应用：

- Iteration 1：基于3-tier参考架构创建表示层、业务层、数据层元素
- Iteration 2：通过动态分析（序列图）定义接口 — **Logic View**
- Iteration 3：实例化可用性战术元素（冗余节点、心跳监控、故障切换）

 Step 5 和 Step 6 可以并行进行。

输入：选定的设计概念

输出：实例化的架构元素及其职责和接口定义

Step 6：画视图草图并记录设计决策

维度	内容
注意	这不是完整文档，而是草图级别的记录
关键实践	使用非正式符号但要保持一致性
核心习惯	养成记录元素职责和设计决策的纪律
为什么要及时记录？	设计时记录，确保以后不需要"回忆"

输入：实例化的元素
输出：架构视图草图 + 设计决策记录

Step 7：分析当前设计、评审迭代目标

关键工具：设计看板（Design Kanban Board） — 跟踪设计进展

分析要点：

- 当前设计是否满足了本轮迭代的驱动因子？
- 整体设计目的是否在持续推进？

输入：当前设计 + 迭代目标 + 设计目的
输出：设计看板更新 + 是否继续迭代的决策

5.5 设计终止条件

ADD 迭代在满足以下任一条件时终止：

条件	理想程度
所有 ASRs 的设计决策都已做出（设计目标达成）	★★★★ 最理想
最重要的技术风险已经得到缓解	★★★
分配给架构设计的时间已用完	★ 不太理想

5.6 ADD 的迭代性质

This may be only a partial architecture design and is not Big Design Up Front (BDUF)!

- 产出可能是部分架构设计
- 不是瀑布式的一步到位，而是敏捷的渐进设计
- 每次迭代只做少量设计决策
- 可以以敏捷方式执行：使用初始文档（草图）+ 设计看板跟踪进展

六、应用ADD到不同系统场景

6.1 三种系统场景

场景	特征	设计策略
成熟领域绿地系统	从零开始，领域知识丰富（Web企业应用、微服务）	有明确的参考架构路线图
新领域绿地系统	从零开始，缺乏既有基础设施和知识	从第一原则出发，使用原型和通用概念
棕地系统	对已有系统进行修改	先理解现有架构，再增量设计

6.2 成熟领域绿地系统的设计路线图

阶段	目标	活动
初始迭代	建立整体系统结构	选择参考架构、模式、框架；评估非功能约束和QA
下一代	识别主要功能结构	将用例分配到元素；分解参考架构元素；规划团队分配
后续迭代	细化剩余驱动因子的结构	使用战术、模式、组件和最佳实践（模块化、低耦合）

路线图的好处：指导初始设计、辅助早期估算。使用已知组件减少风险，从一开始就支持质量目标。

6.3 新领域绿地系统的设计策略

策略	说明
无参考架构起点	没有现成的模型或可复用组件 — 从第一原则开始
使用原型和通用概念	用模式、战术和以质量属性为中心的原型来指导设计
关注新兴挑战	关注性能、可扩展性、安全性；新领域需要灵活的迭代目标

6.4 棕地系统的设计策略

步骤	说明
1. 先理解现有架构	识别系统元素及其交互是分解和迭代计划的前提
2. 评估后再迭代	一旦理解架构后，ADD步骤与绿地设计类似，增量式处理新驱动因子

6.5 替换遗留系统：绞杀者模式（Strangler Fig Pattern）

要素	说明
问题	旧系统依赖过时技术或无法管理的技术债务
方案	引入代理（Proxy）将调用路由到遗留系统，同时逐步替换组件
迭代替换	每个ADD迭代针对特定服务或组件进行替换过渡

七、架构文档化

★ 对应考试 QI-7（6分简答题）：为什么需要不同视图？给出4个示例视图的名称和目的。

7.1 为什么架构文档化很重要？

Even the best architecture will be useless if people do not know what it is, cannot understand it, or misunderstand it.

风险	后果
不知道架构是什么	无法使用和维护
无法理解架构	无法构建或修改
误解架构	错误应用，系统崩溃

7.2 架构文档的三大用途

用途	说明
教育（Education）	向新成员、外部分析师、新架构师介绍系统
沟通（Communication）	涉众之间的主要沟通工具（架构师→开发者、架构师→未来架构师）
分析与构建基础（Basis for Analysis & Construction）	告诉实现者要实现什么；记录未解决问题的容器；架构评估的基础

7.3 文档的两个属性

Architecture documentation is both prescriptive and descriptive.

属性	面向对象	含义
规定性（Prescriptive）	某些受众	规定应该是什么，对尚未做出的决策施加约束
描述性（Descriptive）	其他受众	描述实际是什么，重述已经做出的设计决策

7.4 符号（Notations）的三种级别

级别	特征	优点	缺点
非正式（Informal）	通用绘图工具，自然语言描述语义	容易创建，快速	不能进行正式分析
半正式（Semiformal）	标准化符号，规定图形元素和构造规则	可进行基础分析	缺少完整的语义处理
	UML 属于半正式符号		
正式（Formal）	精确的数学语义	形式化分析（语法+语义）	创建和理解更费时
	如 ADLs (Architecture Description Languages)	支持自动化工具	更少的保证（更少的分析可能性）

⚠️ 选择符号的核心原则：不同类型的符号适合表达不同类型的信息。UML类图不能帮你推理可调度性，时序图不能告诉你系统按时交付的可能性。选择符号时要清楚你需要捕获和推理的关键问题。

7.5 视图（Views）

为什么需要不同视图？

Views let us divide a software architecture into a number of manageable representations of the system.

核心原则：Documenting an architecture is a matter of documenting the relevant views and then adding documentation that applies to more than one view.

翻译：文档化架构 = 文档化相关视图 + 添加跨视图的补充文档。

原因	解释
不同视图服务于不同目标	性能分析需要C&C视图，变更影响分析需要模块视图
单视图无法表达架构的全部信息	架构是复杂的多维人工制品
信息过载控制	每个视图聚焦一个维度
成本效益	只维护有明确涉众需求的视图

7.6 三大视图类型

（一）模块视图（Module Views）

维度	说明
元素	模块（Modules）：提供一组一致职责的软件实现单元
关系	is-part-of（整体-部分）、depends-on（依赖）、is-a（泛化/特化）
约束	不同模块视图可施加不同拓扑约束（如可见性限制）
用途	代码构建蓝图、变更影响分析、增量开发规划、需求追溯、工作分配、预算编制

任何软件架构文档如果不包含至少一个模块视图，几乎不可能是完整的。

常见模块视图：

- 分解视图（Decomposition View）：is-part-of关系，展示系统如何拆分为模块
- 使用视图（Uses View）：depends-on关系，展示模块间依赖
- 分层视图（Layered View）：allowed-to-use关系，控制可见性

（二）组件-连接器视图（C&C Views）

维度	说明
元素	组件（Components）：主要处理单元和数据存储，通过端口交互；连接器（Connectors）：组件间交互的通路，具有角色（接口）
关系	附着（Attachment）：组件端口与连接器角色相关联；接口委托（Interface Delegation）
约束	组件只能附着到连接器（不能直连其他组件）；连接器不能独立出现；只能兼容端口与角色附着
用途	展示系统如何工作；指导开发（运行时元素的结构和行为）；帮助推理运行时的QA（性能、可用性）

UML 与 C&C 视图：

- UML 组件与 C&C 组件匹配良好
- 但 UML 连接器太简单：不能有子结构、属性或行为描述
- 简单 C&C 连接器 → 用 UML 连接器（线）+ stereotype
- 复杂 C&C 连接器 → 用 UML 组件替代，或标注解释

(三) 分配视图 (Allocation Views)

维度	说明
元素	软件元素（有对环境的需求）+ 环境元素（有提供给软件的属性）
关系	<code>allocated-to</code> （分配到）：软件元素映射到环境元素
用途	推理性能、可用性、安全；分配工作到团队；管理并发访问软件版本

常见分配视图：

- 部署视图 (Deployment View)：软件→硬件节点
- 工作分配视图 (Work Assignment View)：模块→开发团队

7.7 四种示例视图速答（考试答案）

视图名称	视图类型	元素	关系	目的
分解视图 (Decomposition)	模块	模块	is-part-of	展示代码组织，支持工作分配和变更影响分析
分层视图 (Layered)	模块	层	allowed-to-use	控制模块依赖方向，管理可见性
C&C视图 (Component-Connector)	C&C	组件+连接器	attachment	展示运行时交互，推理性能和可用性
部署视图 (Deployment)	分配	软件+硬件	allocated-to	展示物理部署拓扑，推理性能和安全性

7.8 质量视图 (Quality Views)

从结构视图中提取相关部分，针对特定涉众或关注点打包：

质量视图	内容
安全视图	安全角色组件、通信方式、安全信息存储、物理安全措施
通信视图	组件间通道、网络通道、QoS参数、并发区域
异常/错误处理视图	错误检测、报告、解决机制
可靠性视图	复制 (Replication)、切换 (Switchover)、时间问题和事务完整性
性能视图	网络流量模型、最大操作延迟等

7.9 选择视图的方法（三步骤）

Step 1：构建涉众/视图表

	分解视图	C&C视图	部署视图	...
开发者	高细节	中等细节	概览	
项目经理	概览	无	中等细节	
运维人员	无	中等细节	高细节	

Step 2：合并视图以减少数量

- 找出"边缘"视图（只需要概览或服务极少数涉众）与更强需求的视图合并
- 自然合并的组合：各C&C视图之间、部署视图+SOA/进程视图、分解视图+工作分配/实现/使用/分层视图

Step 3：排优先级和分阶段

- 分解视图应优先发布（高层分解容易设计，项目经理可开始组建团队和编制预算）
- 不需要一次性满足所有涉众的全部信息需求—80%信息就已足够
- 一个视图完成之前就可以开始另一个—广度优先方法通常最好

7.10 文档化一个视图（View Template）

每个视图的标准文档结构（5个部分）：

节	内容	关键提示
Section 1: 主要展示	视图的元素和关系，通常是图形化的	⚠ 一定要包含图例/符号说明（Key）！缺少图例是实践中最常见的文档错误
Section 2: 元素目录	元素和属性、关系和属性、元素接口、元素行为	如果主要展示中省略了元素或关系，应在此处补充
Section 3: 上下文图	系统在此视图图中与环境的关系	环境实体可以是人、其他计算机系统、物理对象
Section 4: 可变性指南	如何运用此视图中的变化点	
Section 5: 理由	为什么设计成这样，选择的模式为什么比备选方案更好	提供令人信服的论证

7.11 超越视图的文档（Documentation Beyond Views）

节	内容
文档控制信息	组织、版本号、日期、变更历史
Section 1: 文档路线图	范围与概要、文档组织方式、视图概览、涉众使用方式（含场景）
Section 2: 视图文档化方式	说明你使用的标准组织格式
Section 3: 系统概览	系统功能、用户、重要背景或约束的简短描述
Section 4: 跨视图映射	不同视图的元素之间的映射关系表（如"is implemented by""implements""included in"）
Section 5: 跨视图理由	适用于多个视图的架构决策（背景/组织约束、驱动需求、基础架构模式的选择）
Section 6: 目录	索引、术语表、缩略词表

📌 为符合 ISO/IEC 42010-2007，必须至少考虑以下涉众的关注点：用户（users）、采购方（acquirers）、开发者（developers）、维护者（maintainers）。

7.12 文档化行为（Documenting Behavior）

行为文档补充每个视图，描述架构元素如何交互。

两种行为描述语言：

类型	特征	例子
面向轨迹的语言 (Trace-oriented)	描述系统在特定刺激下的 特定活动序列	用例、序列图、通信图、活动图、时序图、消息序列图
全面的语言 (Comprehensive)	展示结构元素的 完整行为 ，可推断从初始到最终的所有路径	状态机（State Machine） — 每个状态是所有可能到达该状态的历史的抽象

7.13 架构变化快于文档更新的应对策略

对于运行时变化或高频发布部署的系统：

- 文档化所有版本的**共同内容**：记录**不变量**，使文档时架构更多是对约束和指南的描述
- 文档化允许的变化方式：通常在**可变性指南**中描述（添加新组件、替换组件实现）

7.14 敏捷开发中的架构文档化

原则	说明
采用模板	用标准组织来捕获设计决策
只为有明确涉众的视图写文档	
按需填写	信息就绪时才写，只写对下游有价值的内容
不必区分架构设计文档和详细设计文档	产出刚好够进入编码的设计信息就好
不需要填满所有模板内容	口头传达的信息写"N/A"
白板图也是文档	拍照，用作主要展示

八、讨论问题与思考

这些是课件最后提出的思考题，很有可能是变体考试题。

Q1：ADD Step 2 中，如果迭代目标范围太大可能发生什么？

答案要点：

- 迭代难以完成，设计决策变得模糊
- 太多驱动因子同时处理 → 设计焦散、无法深入
- 违反了 ADD 的核心哲学：**每次迭代聚焦少量决策**

Q2：为什么成熟领域绿地系统推荐先识别支持主要功能的结构，再处理质量属性？

答案要点：

- 成熟领域有现成的参考架构（如Web应用的三层结构），先搭骨架效率最高
- 功能结构为后续的QA细化提供了**可操作的上下文**（知道在哪里应用战术）
- 如果反过来先处理QA，可能设计出满足QA但在功能上不切实际的架构

Q3：架构师决定对所有用例执行 ADD 迭代会有什么后果？

答案要点：

- 设计过程过长，可能陷入分析瘫痪
- 不是所有用例都是ASR — 大量时间浪费在非关键用例上
- 违反"足够设计"原则 — 不需要为每个用例都做完整的架构设计

Q4：不执行 Step 6（及时记录），把文档化留到以后有什么风险？

答案要点：

- **遗忘设计决策** — 后续需要"回忆"而不是"查阅"
- 设计决策失去可追溯性 — 无法向涉众解释"为什么这样设计"
- 架构评估缺乏基础 — 没有记录就无法系统评估
- 团队沟通效率下降 — 新成员无法通过文档理解系统

附录：ADD 3.0 核心术语表

术语	英文	定义
架构驱动因子	Architectural Driver	塑造架构形状的需求子集，是设计过程的输入
架构显著需求	ASR	对架构有决定性影响的需求
设计概念	Design Concept	解决子问题的已有解决方案（参考架构、模式、战术等）
战术	Tactic	影响质量属性响应控制的原子级设计决策
模式	Pattern	对反复出现问题的验证过的概念性解决方案，由多个战术组成
参考架构	Reference Architecture	提供应用结构化蓝图的模板
部署模式	Deployment Pattern	从物理角度构建系统的指导
框架	Framework	提供通用功能、处理跨应用关注点的可复用软件元素
设计看板	Design Kanban Board	跟踪设计进展的可视化工具
绿地系统	Greenfield System	从零开始的新系统
棕地系统	Brownfield System	对已有系统的修改
绞杀者模式	Strangler Fig Pattern	通过代理逐步替换遗留系统的模式
模块视图	Module View	展示代码组织方式的视图
C&C视图	Component-Connector View	展示运行时组件交互的视图
分配视图	Allocation View	展示软件到环境映射的视图
质量视图	Quality View	从结构视图中提取的、服务于特定QA分析的视图
BDUF	Big Design Up Front	瀑布式一步到位的完整设计（ADD反对这种模式）



延伸阅读：

- *Software Architecture in Practice* (4th ed.) — Bass, Clements, Kazman
- *Application Architecture Guide* — Microsoft Patterns & Practices



关联文档：

- 《质量属性-完整知识点整理.md》 — 7大质量属性的详细战术分类和场景建模
- 《软件架构模式-完整知识点整理.md》 — 9个时代架构演进全景图和每个时代的设计概念
- 《软件体系结构-期末考试高分复习指南.md》 — 全题型覆盖的考试答题指南

整理自：南京大学软件学院《Designing Software Architectures》课件（李杉杉，2026）